# Tarea Nº 4:

**GRUPAL**

## Métodos de Ordenamiento Recursivo: QuickSort y MergeSort

1. **Agencia de Viajes y Gestión de Paquetes**
   Una agencia de viajes está implementando un sistema para gestionar sus destinos, los paquetes de viaje ofrecidos y las ventas realizadas a sus clientes, con el fin de optimizar precios y comisiones.
   - Cree la clase **Destino** con los atributos: Código_Destino (cadena), Nombre_Ciudad, País y Costo_Base_Vuelo (doble).
   - Cree la clase **Paquete_Viaje** con los atributos: ID_Paquete (entero), Nombre_Paquete, Duración_Días (entero) y Precio_Total (doble).
   - Cree la clase **Venta_Paquete** (clase de relación) con los atributos: Paquete_Asignado (objeto de la clase Paquete_Viaje), Destino_Seleccionado (objeto de la clase Destino), Fecha_Venta y Comisión_Agencia (doble, calculada como el 10% del Precio_Total).
   - Implemente una clase controladora o principal que mantenga **arreglos de objetos** para un máximo de **8 Destinos**, **15 Paquetes_Viaje** y **20 Ventas_Paquete**.
   - El programa debe permitir la gestión completa de las Ventas_Paquete (agregar, eliminar, mostrar), garantizando que el Precio_Total del paquete se calcule correctamente al crearlo.
   - Implemente un menú de opciones que permita:
     - Registrar nuevos Destinos y Paquetes de Viaje.
     - Registrar una Venta: Se debe ingresar el ID del paquete y el Código del destino.
     - Mostrar todos los paquetes, ordenados por **QuickSort** según su Precio_Total.
     - Mostrar todas las ventas, ordenadas por **MergeSort** según la Comisión_Agencia.
     - Buscar paquetes por **Código_Destino** (búsqueda que retorne todos los paquetes disponibles para ese destino).
     - Buscar un destino por **Código_Destino** (búsqueda única).
   - Implemente los métodos de ordenamiento **QuickSort** y **MergeSort** y los métodos de búsqueda requeridos.

2. **Control de Préstamos Bancarios y Clientes**
   Un banco requiere un sistema para administrar a sus clientes, registrar los distintos tipos de préstamos que otorga y llevar un seguimiento del saldo pendiente de cada operación crediticia.
   - Cree la clase **Cliente** con los atributos: CI_Cliente (cadena), Nombre_Completo, Ingreso_Mensual (doble) y Total_Deuda_Acumulada (doble, suma de los saldos pendientes de todos sus préstamos).
   - Cree la clase **Tipo_Préstamo** con los atributos: Código_Préstamo (entero), Nombre_Tipo (ej. Hipotecario, Personal, Vehicular), Tasa_Interés_Anual (doble) y Plazo_Máximo_Meses (entero).
   - Cree la clase **Operación_Préstamo** (clase de relación) con los atributos: Cliente_Asignado (objeto de la clase Cliente), Tipo_Asignado (objeto de la clase Tipo_Préstamo), Monto_Inicial (doble) y Saldo_Pendiente (doble).
   - Implemente una clase controladora o principal que maneje **arreglos de objetos** para un máximo de **10 Clientes**, **5 Tipos_Préstamo** y **15 Operaciones_Préstamo**.
   - El programa debe permitir la gestión de las Operaciones_Préstamo (agregar, editar, eliminar, mostrar). Al agregar un préstamo, debe actualizar automáticamente el atributo Total_Deuda_Acumulada del cliente.
   - Implemente un menú de opciones que permita:
     - Registrar Clientes y Tipos de Préstamos.
     - Registrar una Operación_Préstamo: Se debe ingresar el CI del cliente y el Código del tipo de préstamo.
     - Actualizar el Saldo_Pendiente de una operación (ej. tras un abono).
     - Mostrar todos los clientes, ordenados por **QuickSort** según su Total_Deuda_Acumulada.
     - Mostrar todos los préstamos, ordenados por **MergeSort** según el Monto_Inicial.
     - Buscar operaciones de préstamo por **Código_Préstamo** (búsqueda que muestre todos los clientes que tienen ese tipo de crédito).
     - Buscar un cliente por **CI_Cliente** (búsqueda única).
   - Implemente los métodos de ordenamiento **QuickSort** y **MergeSort** y los métodos de búsqueda requeridos.

3. **Estacionamiento Vehicular Inteligente**
   Un estacionamiento grande necesita una aplicación para controlar las zonas disponibles, los vehículos registrados y el tiempo de permanencia para calcular la tarifa a pagar.
   - Cree la clase **Zona_Estacionamiento** con los atributos: Código_Zona (cadena, ej. A1, B2), Nivel (entero), Capacidad_Máxima (entero) y Espacios_Ocupados (entero).

- Cree la clase **Vehículo** con los atributos: Placa (cadena, clave única), Tipo_Vehículo (ej. Auto, Moto, Camioneta) y Hora_Entrada (cadena/tipo hora).
- Cree la clase **Registro_Estacionamiento** (clase de relación) con los atributos: Vehículo_Estacionado (objeto de la clase Vehículo), Zona_Asignada (objeto de la clase Zona_Estacionamiento), Hora_Salida (cadena/tipo hora) y Tarifa_Calculada (doble).
- Implemente una clase controladora o principal que gestione **arreglos de objetos** para un máximo de **12 Zonas_Estacionamiento**, **20 Vehículos** y **30 Registros_Estacionamiento**.
- El programa debe permitir la gestión de los Registros_Estacionamiento (entrada y salida). Al registrar una entrada, debe validar la disponibilidad y actualizar los Espacios_Ocupados de la zona. Al registrar la salida, se debe calcular la Tarifa_Calculada (asumiendo una tarifa simple: 1 hora = $5, y se cobra por hora completa).
- Implemente un menú de opciones que permita:
  - Registrar Zonas y la entrada de un Vehículo (se asigna a una zona con espacio).
  - Registrar la Salida de un Vehículo: Se debe ingresar la Placa, calcular el tiempo, la tarifa y liberar el espacio en la zona.
  - Mostrar todos los vehículos (con registro de entrada), ordenados por **QuickSort** según la Hora_Entrada.
  - Mostrar todas las zonas, ordenadas por **MergeSort** según el Espacios_Ocupados (de menor a mayor ocupación).
  - Buscar registros de estacionamiento (historial) por **Tipo_Vehículo** (búsqueda que muestre todas las veces que un tipo de vehículo usó el estacionamiento).
  - Buscar una zona por **Código_Zona** (búsqueda única, mostrando su estado actual).
- Implemente los métodos de ordenamiento **QuickSort** y **MergeSort** y los métodos de búsqueda requeridos.

**INSTRUCCIONES PARA EL ENVÍO**

Cree una carpeta **GrupoXX_Tarea_04** y dentro de ella coloque cada uno de los ejercicios creados. Únicamente comprima a la carpeta (no comprima los ejercicios). Luego envíe la carpeta comprimida.